

Telecom Database Normalization

Complete Assignment Answers (1NF to 5NF)

Original Telecom Table (Reference)

The telecom company stores all customer subscription data in one flat table:

cust_id	customer_name	phone_number	plan_name	plan_price	services	devices	tower_id	tower_location
C001	Aung Aung	09123456789	Premium	30000	Voice, SMS, Data	iPhone, Router	T01	Yangon
C002	Su Su	09987654321	Basic	10000	Voice, SMS	Samsung	T02	Mandalay
C003	Kyaw Kyaw	09777777777	Premium	30000	Voice, SMS, Data	Router	T01	Yangon

Q1. What normalization problems exist in the original telecom table?

The original table violates multiple normal forms simultaneously:

- Non-Atomic (Multi-valued) Columns — 1NF Violation: The 'services' column stores multiple values (e.g., 'Voice, SMS, Data') and the 'devices' column also stores multiple values (e.g., 'iPhone, Router'). Each cell must hold only one atomic value.
- Partial Dependencies — 2NF Violation: With a composite key of (customer_id, service, device), attributes like customer_name, phone_number, plan_name, and plan_price depend only on customer_id, not the full composite key.
- Transitive Dependencies — 3NF Violation: plan_price depends on plan_name (not directly on customer_id): customer_id -> plan_name -> plan_price. Similarly, tower_location depends on tower_id: customer_id -> tower_id -> tower_location.
- Multi-valued Dependencies — 4NF Violation: A customer's services and devices are completely independent of each other. Storing them together creates unnecessary combinatorial row explosion.
- Data Redundancy: Customer details, plan info, and tower location are duplicated across every row for each service/device combination. This causes insertion, update, and deletion anomalies.

Q2. Convert the telecom table into 1NF

First Normal Form (1NF) requires:

- Each column contains only atomic (single, indivisible) values
- Each row is uniquely identifiable
- No repeating groups or set-valued attributes

Resolution: Expand multi-valued 'services' and 'devices' into separate rows. The composite primary key becomes (customer_id, service, device).

cust_id	cust_name	phone_no	plan_name	plan_price	service	device	tower_id	tower_location
C001	Aung Aung	09123456789	Premium	30000	Voice	iPhone	T01	Yangon
C001	Aung Aung	09123456789	Premium	30000	Voice	Router	T01	Yangon
C001	Aung Aung	09123456789	Premium	30000	SMS	iPhone	T01	Yangon
C001	Aung Aung	09123456789	Premium	30000	SMS	Router	T01	Yangon
C001	Aung Aung	09123456789	Premium	30000	Data	iPhone	T01	Yangon
C001	Aung Aung	09123456789	Premium	30000	Data	Router	T01	Yangon
C002	Su Su	09987654321	Basic	10000	Voice	Samsung	T02	Mandalay
C002	Su Su	09987654321	Basic	10000	SMS	Samsung	T02	Mandalay
C003	Kyaw Kyaw	09777777777	Premium	30000	Voice	Router	T01	Yangon
C003	Kyaw Kyaw	09777777777	Premium	30000	SMS	Router	T01	Yangon
C003	Kyaw Kyaw	09777777777	Premium	30000	Data	Router	T01	Yangon

The composite primary key is (customer_id, service, device) to ensure row uniqueness. Note the significant row multiplication this causes.

Q3. What is the main issue after converting to 1NF?

After converting to 1NF, the table suffers from Partial Dependencies, violating 2NF. With a composite primary key of (customer_id, service, device):

- customer_name depends only on customer_id — not on service or device
- phone_number depends only on customer_id
- plan_name depends only on customer_id
- plan_price depends only on customer_id
- tower_id depends only on customer_id
- tower_location depends only on customer_id

This means customer details like Aung Aung's plan and tower are repeated in 6 rows (3 services x 2 devices). Any update to a plan price must be applied across all matching rows, risking inconsistency.

Q4. Convert the 1NF table into 2NF

Second Normal Form (2NF) requires: The table is in 1NF AND every non-key attribute fully depends on the entire primary key.

Resolution: Remove partial dependencies by decomposing into three tables.

Table 1: Customer (Primary Key: customer_id)

cust_id (PK)	cust_name	phone_number	plan_name	plan_price	tower_id	tower_location
C001	Aung Aung	09123456789	Premium	30000	T01	Yangon
C002	Su Su	09987654321	Basic	10000	T02	Mandalay
C003	Kyaw Kyaw	09777777777	Premium	30000	T01	Yangon

Table 2: Customer_Service (Primary Key: customer_id + service)

cust_id (FK)	service	
C001	Voice	
C001	SMS	
C001	Data	

C002	Voice	
C002	SMS	
C003	Voice	
C003	SMS	
C003	Data	

Table 3: Customer_Device (Primary Key: customer_id + device)

cust_id (FK)	device	
C001	iPhone	
C001	Router	
C002	Samsung	
C003	Router	

All non-key attributes in each table now fully depend on that table's complete primary key.

Q5. What issue still exists after 2NF?

After achieving 2NF, the Customer table still contains Transitive Dependencies, violating 3NF:

- customer_id -> plan_name (direct dependency)
- plan_name -> plan_price (plan_price depends on plan_name, not customer_id)
- Therefore: customer_id -> plan_name -> plan_price (transitive dependency)
- customer_id -> tower_id (direct dependency)
- tower_id -> tower_location (location depends on tower, not customer)
- Therefore: customer_id -> tower_id -> tower_location (transitive dependency)

Consequence: If 'Premium' plan changes its price, every customer row on the Premium plan must be updated. Two customers (C001, C003) both use T01/Yangon — this location is stored twice and must be kept in sync.

Q6. Convert the 2NF design into 3NF

Third Normal Form (3NF) requires: The table is in 2NF AND no non-key attribute is transitively dependent on the primary key.

Resolution: Extract Plan and Tower into their own tables, replacing the transitive attributes with foreign keys.

Table 1: Customer (Primary Key: customer_id)

cust_id (PK)	cust_name	phone_number	plan_id (FK)	tower_id (FK)	
C001	Aung Aung	09123456789	PL01	T01	
C002	Su Su	09987654321	PL02	T02	
C003	Kyaw Kyaw	09777777777	PL01	T01	

Table 2: Plan (Primary Key: plan_id)

plan_id (PK)	plan_name	plan_price	
PL01	Premium	30000	
PL02	Basic	10000	

Table 3: Tower (Primary Key: tower_id)

tower_id (PK)	tower_location	
T01	Yangon	
T02	Mandalay	

Customer_Service and Customer_Device tables remain unchanged from 2NF. Now every non-key attribute depends directly and only on its table's primary key.

Q7. Is the telecom database now in BCNF?

Boyce-Codd Normal Form (BCNF) requires: For every functional dependency $X \rightarrow Y$, X must be a superkey.

Assessment: Yes, the telecom database satisfies BCNF after the 3NF conversion:

- Customer table: customer_id (superkey) $\rightarrow$ customer_name, phone_number, plan_id, tower_id. The only determinant is the superkey.
- Plan table: plan_id (superkey) $\rightarrow$ plan_name, plan_price. Satisfied.
- Tower table: tower_id (superkey) $\rightarrow$ tower_location. Satisfied.
- Customer_Service: (customer_id, service) composite superkey is the sole determinant. Satisfied.
- Customer_Device: (customer_id, device) composite superkey is the sole determinant. Satisfied.

Potential BCNF concern: If phone_number is unique per customer, then phone_number $\rightarrow$ customer_id is also a functional dependency. Since phone_number would be a candidate key, it is a superkey, so BCNF is still satisfied. If plan_name were unique and plan_name $\rightarrow$ plan_id existed, that is also a candidate key relationship and BCNF holds.

Q8. What 4NF issue can exist in telecom data?

Fourth Normal Form (4NF) requires: The table is in BCNF AND there are no non-trivial multi-valued dependencies (MVDs).

The 4NF issue in telecom data arises because a customer's services and devices are completely independent of each other:

- customer_id $\twoheadrightarrow$ service (a customer multi-determines services independently of devices)
- customer_id $\twoheadrightarrow$ device (a customer multi-determines devices independently of services)

If both were stored in one table, every combination of service and device would need a row. For C001 with 3 services and 2 devices, this means 6 rows (3x2) even though only 2 devices and 3 services exist independently:

- C001 | Voice | iPhone
- C001 | Voice | Router
- C001 | SMS | iPhone
- C001 | SMS | Router
- C001 | Data | iPhone
- C001 | Data | Router

Resolution (already applied in 2NF): Customer_Service and Customer_Device are stored as separate tables. This eliminates the multi-valued dependency and achieves 4NF.

Q9. What is a possible 5NF issue in telecom data?

Fifth Normal Form (5NF), also called Project-Join Normal Form (PJNF), requires: The table cannot be losslessly decomposed further — no join dependencies exist beyond those implied by candidate keys.

A possible 5NF issue in telecom data involves a three-way relationship between customers, services, and devices. Consider this business rule:

'A customer can use a particular service on a particular device, but not every service is available on every device they own.'

Example:

- C001 uses Voice on iPhone
- C001 uses Data on Router
- C001 does NOT use Data on iPhone (no mobile data plan on that device)

In this case, the relationship is not simply the cross-product of (customer x service) and (customer x device). A three-way join dependency exists between customer, service, and device that cannot be losslessly decomposed into any two-way tables.

Resolution: Create a Customer_Service_Device junction table with composite key (customer_id, service, device) to explicitly track which services are used on which devices per customer, ensuring no 5NF violation.

Q10. What is the final normalized telecom database design?

The final fully normalized telecom database (satisfying 1NF through 5NF) consists of the following tables:

Table	Primary Key	Columns
Customer	customer_id	customer_name, phone_number, plan_id (FK), tower_id (FK)
Plan	plan_id	plan_name, plan_price
Tower	tower_id	tower_location
Customer_Service	customer_id + service	customer_id (FK), service
Customer_Device	customer_id + device	customer_id (FK), device
Customer_Service_Device (5NF opt.)	cust_id + service + device	customer_id (FK), service, device

Summary of Each Table

1. Customer Table (customer_id PK)

- Stores core customer identity and contact information
- References Plan via plan_id (FK) and Tower via tower_id (FK)
- All transitive dependencies removed in 3NF

2. Plan Table (plan_id PK)

- Resolves transitive dependency: plan_name -> plan_price
- Plan price is stored exactly once — changing a plan price updates one row
- Supports adding new plan attributes (data_limit, speed) without affecting Customer table

3. Tower Table (tower_id PK)

- Resolves transitive dependency: tower_id -> tower_location
- Tower location stored once — multiple customers on same tower need no duplication
- Supports adding tower attributes (capacity, technology) cleanly

4. Customer_Service Table (customer_id + service composite PK)

- Resolves partial dependency from 2NF conversion
- Resolves 4NF multi-valued dependency — services are independent of devices
- A customer can subscribe to any number of services

5. Customer_Device Table (customer_id + device composite PK)

- Resolves partial dependency from 2NF conversion
- Resolves 4NF multi-valued dependency — devices are independent of services
- A customer can register any number of devices

6. Customer_Service_Device Table (cust_id + service + device composite PK) — 5NF optional

- Addresses 5NF join dependency when service-device pairing per customer matters
- Tracks exactly which services are used on which devices per customer
- Use when business rules dictate service-device specific constraints

Normalization Journey Summary

Normal Form	Requirement	Action Taken
1NF	Atomic values, no repeating groups	Split multi-valued services and devices into separate rows; composite PK = (cust_id, service, device)

2NF	No partial dependencies	Moved customer info to Customer table; created Customer_Service and Customer_Device tables
3NF	No transitive dependencies	Extracted Plan table (plan_name->plan_price) and Tower table (tower_id->location)
BCNF	Every determinant is a superkey	Satisfied — all determinants are superkeys in the final schema
4NF	No multi-valued dependencies	Separate Customer_Service and Customer_Device tables eliminate independent MVDs
5NF	No join dependencies	Optional Customer_Service_Device table for three-way relationship